

# Star Filters

## Reporte de Implementación

### Lo que ya está bien hecho

UX/UI · SEO · Ciberseguridad

Este documento explica, de forma muy sencilla (como si se lo contaras a un niño de 5 años), todo lo que el sitio web YA tiene bien implementado en tres áreas importantes: que sea fácil y bonito de usar (UX/UI), que Google lo encuentre (SEO) y que esté protegido (Ciberseguridad).

Solo se incluye lo que ya está hecho y funcionando. Cada punto tiene una explicación fácil y, debajo, el detalle técnico para tu equipo.

Proyecto: StarFilters E-commerce · Fecha: 05/06/2026

Tecnología: Astro (SSR) · TypeScript · MySQL · Stripe · BIND ERP

¿Qué vas a encontrar en este reporte?

Imagina que tu sitio web es una tienda de verdad. Para que funcione bien necesita tres cosas: que la gente entre a gusto y encuentre todo rápido (eso es UX/UI), que haya muchos letreros en la calle para que la gente sepa que existe (eso es SEO), y que tenga buenas cerraduras y cámaras para que nadie robe (eso es Ciberseguridad). Tu tienda ya tiene muchas de esas cosas bien puestas. Aquí te las contamos una por una.

- 1

**UX/UI**

Que el sitio sea fácil, rápido y bonito de usar.
- 2

**SEO**

Que Google y la gente encuentren el sitio fácilmente.
- 3

**Ciberseguridad**

Que la información y el dinero estén protegidos.

*Nota: este reporte describe únicamente funciones que ya existen en el código del proyecto al día de hoy. No incluye sugerencias ni pendientes.*

# 1 UX / UI — Que sea fácil y bonito de usar

UX/UI significa que las personas usen el sitio sin esfuerzo y que se vea bien en el celular y en la computadora.

## Navegación y estructura

### ✓ Menú (Navbar) y pie de página en todas las páginas

*En simple:* Arriba siempre hay un menú para moverte y abajo un pie con la información importante, igual en toda la tienda.

*Cómo está hecho:* Layout central 'websiteLayout.astro' que incluye <Navbar>, <FooterSection> y botón flotante de WhatsApp en cada página, garantizando una experiencia consistente.

### ✓ Botón de WhatsApp siempre a la mano

*En simple:* Hay un botón flotante para escribir por WhatsApp desde cualquier parte, sin tener que buscarlo.

*Cómo está hecho:* Componente 'WhatsAppButton.astro' fijo en el layout, disponible en todas las vistas del sitio público.

### ✓ Transiciones suaves entre páginas

*En simple:* Cuando cambias de página no 'parpadea' todo; se siente fluido como una app.

*Cómo está hecho:* Uso de <ClientRouter /> (View Transitions de Astro) en el layout para navegación tipo SPA con transiciones nativas.

### ✓ Animaciones de aparición al hacer scroll

*En simple:* Los elementos aparecen suavemente cuando bajas, lo que hace el sitio más vivo y agradable.

*Cómo está hecho:* Componente 'RevealOnScroll.astro' aplicado de forma uniforme en todo el sitio (reveal on scroll con IntersectionObserver).

## Velocidad y rendimiento

### ✓ Las imágenes se cargan solo cuando se necesitan (lazy loading)

*En simple:* Las fotos que están más abajo no se descargan hasta que te acercas, así la página abre más rápido.

*Cómo está hecho:* Componente 'LazyImage.astro' con loading='lazy', decoding='async', placeholder en base64 y carga 'eager' solo para imágenes críticas (above the fold).

### ✓ Renderizado en el servidor (SSR)

*En simple:* La página llega ya 'armada' desde el servidor, por eso se ve rápido y completa desde el primer momento.

*Cómo está hecho:* Astro configurado con output: 'server' y adaptador @astrojs/node en modo middleware sobre Express.

### ✓ Caché de productos

*En simple:* El sitio recuerda los productos un ratito para no tener que pedirlos una y otra vez, y así responder más rápido.

*Cómo está hecho:* Módulo 'product-cache.ts' para evitar consultas repetidas a la base de datos / ERP.

### ✓ Imágenes optimizadas con Cloudinary

*En simple:* Las fotos se guardan y entregan ya optimizadas (peso y tamaño justos) para que carguen ligeras.

*Cómo está hecho:* Integración con Cloudinary ('cloudinary.ts' + sharp) para alojar y transformar imágenes.

## Diseño responsivo y accesibilidad

### ✓ Se ve bien en celular, tablet y computadora

*En simple:* No importa con qué aparato entres: el sitio se acomoda solo para verse bien.

Cómo está hecho: Diseño responsivo con Tailwind CSS v4 y meta viewport configurado; el layout usa utilidades responsivas en todos los componentes.

### ✓ Textos alternativos en imágenes (alt)

*En simple:* Cada imagen tiene una descripción escondida que ayuda a personas con lectores de pantalla y a Google.

Cómo está hecho: Atributo alt presente en componentes de imagen (LazyImage exige 'alt' obligatorio) y a lo largo de páginas y componentes.

### ✓ Etiquetas de accesibilidad (ARIA)

*En simple:* Algunos botones y menús 'le avisan' a los lectores de pantalla qué son, para personas con discapacidad visual.

Cómo está hecho: Uso de atributos aria-\* en componentes interactivos (menús, botones, modales basados en Radix UI, que ya trae accesibilidad incorporada).

### ✓ Componentes accesibles de Radix UI

*En simple:* Las ventanitas, avisos y globos de ayuda ya vienen pensados para todos, incluso con teclado.

Cómo está hecho: Uso de @radix-ui (dialog, alert-dialog, tooltip, separator, slot) que cumplen prácticas de accesibilidad (foco, teclado, roles ARIA).

## Idiomas y experiencia de compra

### ✓ Sitio en español e inglés

*En simple:* La tienda habla dos idiomas: si llega alguien de otro país, puede verla en inglés.

Cómo está hecho: i18n nativo de Astro (locales 'es' y 'en', defaultLocale 'es'), rutas /en/\* y helpers 'i18n-helpers.ts' para nombres, descripciones y precios por idioma.

### ✓ Precios en MXN y USD

*En simple:* Los precios se muestran en pesos o en dólares según corresponda, sin confundir al cliente.

Cómo está hecho: Servicio 'currency-service.ts' y 'getProductPriceAndCurrency' / 'formatPriceByCurrency' para mostrar precio y moneda correctos.

### ✓ Carrito de compras y proceso de pago claro

*En simple:* Puedes guardar productos en un carrito y pagar paso a paso sin perderte.

Cómo está hecho: Módulo 'cart.ts', páginas /cart y /checkout, y pago con Stripe (@stripe/react-stripe-js) integrado.

### ✓ Confirmación de compra con celebración

*En simple:* Cuando compras, aparece una pantalla de '¡felicidades!' con confeti para que sepas que todo salió bien.

Cómo está hecho: Página /purchase-success con canvas-confetti; refuerza el feedback positivo al usuario tras el pago.

### ✓ Correos automáticos al cliente

*En simple:* Después de comprar o registrarte, te llega un correo bonito confirmando todo.

Cómo está hecho: Servicio 'email.ts' con Resend para correos transaccionales (confirmaciones, verificación, recuperación de contraseña).

## 2 SEO — Que te encuentren en internet

SEO son todos los 'letreros' y pistas que ayudan a Google a entender tu sitio y mostrarlo a más personas cuando buscan.

### Etiquetas para Google y redes sociales

#### ✓ **Componente central de SEO en cada página**

*En simple: Cada página le 'cuenta' a Google su título y de qué trata, de forma ordenada y automática.*

Cómo está hecho: Componente 'SEO.astro' reusable que genera title, meta description, keywords, canonical y robots de forma consistente.

#### ✓ **Títulos y descripciones optimizados**

*En simple: Los títulos no son ni muy largos ni muy cortos; tienen la medida ideal para verse completos en Google.*

Cómo está hecho: Utilidades en 'seo-utils.ts': generateSEOTitle (máx ~60 caracteres con sufijo de marca) y generateSEODescription (máx 160).

#### ✓ **Tarjetas bonitas al compartir (Open Graph y Twitter)**

*En simple: Si alguien comparte un enlace en Facebook o X, aparece una tarjeta con foto, título y descripción.*

Cómo está hecho: Meta tags Open Graph (og:title, og:description, og:image 1200x630, og:locale es\_MX/en\_US) y Twitter Card 'summary\_large\_image' en SEO.astro.

#### ✓ **Imagen por defecto para compartir**

*En simple: Aunque una página no tenga foto propia, siempre se comparte con una imagen oficial de la marca.*

Cómo está hecho: Fallback a '/og-image.jpg' cuando no se especifica ogImage.

### Datos estructurados (Schema.org)

#### ✓ **Google entiende que eres una empresa**

*En simple: Le dices a Google 'soy una empresa, esta es mi info', y puede mostrar datos especiales en los resultados.*

Cómo está hecho: generateOrganizationSchema: JSON-LD tipo Organization con nombre, logo, fundación (1984), país, idiomas y área de servicio.

#### ✓ **Los productos se muestran como productos**

*En simple: Google sabe el precio y si hay existencia, y puede mostrar estrellas/precio en los resultados.*

Cómo está hecho: generateProductSchema: JSON-LD tipo Product con offers, priceCurrency MXN, price, availability (InStock/OutOfStock) y priceValidUntil.

#### ✓ **Los artículos del blog se muestran como artículos**

*En simple: Cada nota del blog le dice a Google quién la escribió y cuándo, para salir mejor posicionada.*

Cómo está hecho: generateArticleSchema: JSON-LD tipo Article con headline, author, publisher, datePublished y dateModified.

#### ✓ **Migas de pan (breadcrumbs) y preguntas frecuentes**

*En simple: Google muestra la 'ruta' para llegar a una página y puede mostrar tus preguntas frecuentes directo en la búsqueda.*

Cómo está hecho: generateBreadcrumbSchema (BreadcrumbList) y generateFAQSchema (FAQPage) disponibles para enriquecer resultados.

## Direcciones limpias e idiomas

### ✓ Direcciones (URLs) bonitas y entendibles

*En simple: Las direcciones se leen fácil (con palabras, no códigos raros), y eso le gusta a Google.*

Cómo está hecho: generateSlug normaliza texto a slugs SEO-friendly (minúsculas, sin acentos, con guiones).

### ✓ Etiqueta canónica (sin contenido duplicado)

*En simple: Le dices a Google cuál es la dirección 'oficial' de cada página para que no se confunda con copias.*

Cómo está hecho: <link rel='canonical'> generado en SEO.astro a partir de la URL actual o de un canonical explícito.

### ✓ Señales de idioma (hreflang)

*En simple: Le avisas a Google qué páginas están en español y cuáles en inglés, para mostrar la correcta a cada quien.*

Cómo está hecho: Etiquetas hreflang y og:locale:alternate generadas según alternateLanguages en SEO.astro.

## Mapa del sitio e indexación

### ✓ Mapa del sitio (sitemap.xml) automático

*En simple: Es como darle a Google una mapa con TODAS tus páginas para que no se pierda ninguna.*

Cómo está hecho: 'sitemap.xml.ts' genera dinámicamente el sitemap con páginas estáticas (ES/EN), productos y posts del blog, con priority y changefreq.

### ✓ Archivo robots.txt bien configurado

*En simple: Le dices a los robots de Google a dónde sí pueden entrar y a dónde no (como admin o carrito).*

Cómo está hecho: 'robots.txt' permite páginas públicas, bloquea /admin/, /api/, login, checkout, etc., y declara la ubicación del sitemap.

### ✓ Bloqueo de indexación en pruebas (staging)

*En simple: Cuando el sitio está en modo prueba, se 'esconde' de Google para no mostrar cosas a medio hacer.*

Cómo está hecho: Variable PUBLIC\_NOINDEX fuerza noindex/nofollow en todo el sitio desde 'websiteLayout.astro'.

### ✓ Favicons y manifiesto (instalable como app)

*En simple: El sitio tiene su iconito en la pestaña del navegador y puede 'instalarse' como una app en el celular.*

Cómo está hecho: Favicons en varios tamaños, apple-touch-icon, site.webmanifest, theme-color y meta tags apple-mobile-web-app en el layout.

## 3 Ciberseguridad — Que todo esté protegido

*Ciberseguridad es tener buenas cerraduras: proteger las contraseñas, los pagos y los datos de los clientes para que nadie haga trampa.*

### Contraseñas y cuentas

#### ✓ Las contraseñas se guardan cifradas (nunca en texto)

*En simple: La contraseña no se guarda como la escribiste; se convierte en un código secreto imposible de leer.*

Cómo está hecho: Hash con bcrypt usando SALT\_ROUNDS = 12 (hashPassword/verifyPassword en 'auth.ts'); jamás se almacena la contraseña en claro.

#### ✓ Reglas de contraseña fuerte

*En simple: Para registrarte, tu contraseña debe tener mayúscula, minúscula, número y mínimo 8 letras: más difícil de adivinar.*

Cómo está hecho: validatePassword exige longitud  $\geq 8$  y presencia de mayúscula, minúscula y dígito antes de aceptar el registro.

#### ✓ Validación de datos de registro y login

*En simple: Antes de crear una cuenta, el sistema revisa que el correo, el nombre y el teléfono estén bien escritos.*

Cómo está hecho: validateEmail, validatePhone, validateRegisterData y validateLoginData validan el formato de los datos de entrada.

#### ✓ Tokens seguros para verificar correo y recuperar contraseña

*En simple: Cuando recuperas tu contraseña, se crea una llave secreta única que no se puede adivinar.*

Cómo está hecho: generateVerificationToken/generateResetToken usan crypto.randomBytes(32) (32 bytes aleatorios criptográficos).

### Sesiones y accesos

#### ✓ Sesión protegida con cookie segura (JWT)

*En simple: Cuando inicias sesión, tu 'pase de entrada' se guarda de forma que un ladrón no lo pueda robar fácil.*

Cómo está hecho: JWT firmado (jsonwebtoken) guardado en cookie con httpOnly: true, sameSite: 'lax', secure en HTTPS y maxAge de 7 días; httpOnly impide acceso desde JavaScript (protege ante XSS robo de cookie).

#### ✓ El pase de entrada caduca solo

*En simple: Tu sesión no dura para siempre: a la semana se cierra sola por seguridad.*

Cómo está hecho: JWT\_EXPIRES\_IN = '7d'; el token expira y verifyJWT rechaza tokens vencidos.

#### ✓ Zona de administración protegida

*En simple: Solo los administradores aprobados pueden entrar al panel de control; a los demás se les saca.*

Cómo está hecho: requireAdmin/isAdminAsync verifican el JWT y consultan en base de datos si el usuario es admin antes de dar acceso a /admin.

#### ✓ Páginas privadas requieren iniciar sesión

*En simple: Tu perfil, tus pedidos y el checkout no se pueden ver sin haber entrado a tu cuenta.*

Cómo está hecho: Helpers requireAuth/authMiddleware redirigen a /login si no hay usuario autenticado, protegiendo páginas como /profile y /orders.

## Pagos e integridad de datos

### ✓ Pagos seguros con Stripe (no guardas tarjetas)

*En simple: El dinero lo maneja Stripe, una empresa experta en pagos; tu sitio nunca guarda los números de tarjeta.*

Cómo está hecho: Integración con Stripe; los datos de tarjeta viajan directo a Stripe vía `@stripe/react-stripe-js`, fuera del alcance de tu servidor.

### ✓ Verificación de firma en los avisos de pago (webhooks)

*En simple: Cuando Stripe avisa que un pago se hizo, el sitio comprueba que el aviso es de verdad y no un engaño.*

Cómo está hecho: `stripe-webhook.ts` valida la cabecera `'stripe-signature'` con `constructEvent` y `STRIPE_WEBHOOK_SECRET`; rechaza eventos sin firma válida.

### ✓ El stock se valida contra el ERP antes de cobrar

*En simple: Antes de cobrar, el sistema revisa en el inventario real que sí haya producto, para no vender de más.*

Cómo está hecho: `create-payment-intent.ts` y `check-stock.ts` leen inventario de BIND ERP (con respaldo en BD local) antes de crear el cobro.

### ✓ Consultas a la base de datos protegidas (sin inyección SQL)

*En simple: Cuando se busca algo en la base de datos, se hace de forma blindada para que nadie 'cuele' comandos maliciosos.*

Cómo está hecho: Uso sistemático de consultas parametrizadas (prepared statements con placeholders '?') en `'database.ts'` vía `mysql2`, evitando inyección SQL.

## Protección de archivos y configuración

### ✓ Las llaves secretas no están en el código

*En simple: Las contraseñas del sistema y llaves de pago se guardan aparte, no a la vista en el código.*

Cómo está hecho: Uso de variables de entorno (`dotenv`) para `JWT_SECRET`, `STRIPE_SECRET_KEY`, `BIND_TOKEN`, credenciales de BD, etc.

### ✓ Carpetas sensibles bloqueadas para robots

*En simple: Las carpetas de administración, scripts y base de datos están escondidas de los buscadores.*

Cómo está hecho: `robots.txt` bloquea `/admin/`, `/api/`, `/scripts/`, `/database/` y archivos `.json`; no aparecen en resultados de búsqueda.

### ✓ Validación de archivos subidos (imágenes)

*En simple: Cuando se suben fotos, pasan por un servicio confiable que las revisa y guarda de forma segura.*

Cómo está hecho: Subida de imágenes gestionada por Cloudinary y utilidades `'image-utils.ts'/'video-utils.ts'`, en lugar de guardar archivos crudos en el servidor.



## Resumen para llevar

---

Tu tienda en línea ya tiene cimientos muy sólidos en las tres áreas clave:

### UX/UI

El sitio es rápido, se ve bien en cualquier aparato, está en dos idiomas, tiene animaciones suaves, carrito, pagos claros y avisos por correo. La gente lo usa sin esfuerzo.

### SEO

Google puede encontrar y entender el sitio: tiene títulos optimizados, tarjetas para redes sociales, datos estructurados de empresa/productos/blog, mapa del sitio automático y robots.txt bien configurado.

### Ciberseguridad

Las contraseñas están cifradas, las sesiones usan cookies seguras, los pagos pasan por Stripe con verificación de firma, las consultas a la base de datos están blindadas contra inyección SQL y las llaves secretas no están en el código.

---

*Este reporte refleja únicamente funcionalidades ya implementadas y verificadas en el código del proyecto StarFilters E-commerce a la fecha indicada en la portada. Está pensado para compartirse con personas técnicas y no técnicas por igual.*